
Twenty-Eight Ways to Measure Nothing: A Failure Catalogue from Instrumenting an Agent Harness

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 An OS layer for agentic AI will standardise checkpointing, recovery, and observ-
2 ability, and those designs can only be compared if they can be measured. We built
3 the instrument such comparison requires — an observational timeline committed
4 to git, exhaustive replay of held-out tests at every observation, detection attributed
5 by coverage — and we report two things.

6 First, the instrument’s failure record: ****twenty-eight defects, of which twenty-three**
7 **produced a plausible number rather than an error, and all but one were present**
8 **while a green test suite watched.**** They collapse into four mechanisms — ambient-
9 environment dependence, infrastructure conflated with measurement, final-state
10 measurement of event-shaped quantities, and output consumed without asserting
11 its producer — and we show each mechanism operating in public infrastructure,
12 including the official SWE-bench grader. The capstone: after nineteen fixes, every
13 validation gate we had passed, and the headline number was still wrong, because
14 the agent executed on the host while every check validated the measurement path.
15 A gate certifies only the paths it exercises.

16 Second, what the corrected instrument measures. On the same benchmark instances,
17 under the same harness and recovery policy, one frontier model stack produced
18 **zero regression events across 40 runs** while another produced **140, in 11 distinct**
19 **breakage incidents across 6 bearing runs** (equal-N Fisher $p = 0.011$) — every
20 recovered event erased by the harness’s own rollback and invisible in the final
21 tree. One officially-*resolved* patch carried three of them, two undetected even by
22 the harness that produced it. Regressions during agent runs are a property of the
23 agent regime, not the benchmark; the final state, which is all existing evaluation
24 examines, is where the evidence is not.

25 1 Introduction

26 The agentic-systems community is converging on an OS layer: shared abstractions for memory,
27 scheduling, checkpointing, recovery. Choosing between abstractions requires evidence, and this
28 paper is about how much harder producing that evidence is than it looks — not in the statistics, but in
29 the plumbing that produces the numbers the statistics consume.

30 Our motivating question is narrow. When a step in a long-horizon agent run fails verification,
31 deployed harnesses either **repair in place** (hand the model its own broken tree) or **roll back and**
32 **retry** (reset to the last verified checkpoint). The second discards work and repays cold-cache token
33 prices; its defence is that it leaves less collateral damage. That claim is testable — if you can say, for
34 every run, *when* previously-working behaviour stopped working.

35 **Contributions.** (i) The instrument: observational checkpointing on a hidden git ref, exhaustive replay,
36 coverage-based attribution, and a validation regime whose controls include re-measuring archived
37 runs. (ii) A twenty-eight-defect catalogue with the mechanism for each, in a four-class taxonomy
38 whose classes we show operating in public harnesses. (iii) First measurements from the corrected
39 pipeline: on a 10-instance development prefix, matched across two frontier stacks at equal budgets,
40 a zero-vs-57-event contrast in which every event was invisible to final-state evaluation — with the
41 recovery-policy contrast pre-declared and running at submission time.

42 2 What has to be measured

43 A **regression event** is a held-out test that passed at some observation of a run and fails at a later one.
44 Three properties make it hard to measure honestly.

45 **It is an event, not a state.** A regression introduced and then repaired leaves no trace at the end of the
46 run — and repair is exactly what a recovery policy does. An instrument that inspects the final artifact,
47 which is how regressions on this benchmark are measured in published work [3], records zero by
48 construction for every recovered event. §5.1 measures how much that misses: in our data, all of it.

49 **Its absence and the instrument's death are the same observation.** Zero is what a clean run
50 produces — and what a probe that cannot execute, a parser that matched nothing, and an empty
51 timeline produce. §4 is twenty-eight instances of this property; the design rules in §4.2–4.3 are what
52 we found sufficient to distinguish the two.

53 **Detection must be attributed, not co-located.** "The harness failed something while the regression
54 was open" credits whichever policy fails most often with the best detection. Attribution needs a causal
55 join: a failing harness check and the broken held-out test must exercise a file the agent changed. We
56 report three levels — attributed detection, co-occurrence (an over-count of detection), and unknown
57 — and never let "could not measure" render as any of them.

58 3 The instrument, and how it is validated

59 **Observational checkpointing.** Every mutating tool call commits the tree to a git ref the agent cannot
60 enumerate, from a private index, so the agent's own `git status` and `git diff` are byte-identical
61 with the instrument on or off. Rollbacks and run-end are observed too — the recovering policy's
62 recoveries must be visible to the timeline that scores it.

63 **Exhaustive replay.** Held-out tests are replayed at *every* observation inside the instance's pinned
64 container. Bisection would assume monotone verdicts; non-monotonicity is the treatment (row C1).

65 **Attribution by coverage,** built once per instance at the base commit, identical across arms by
66 construction.

67 **Validation.** Five pre-set gates (negative control, positive control including *recovered* injections, flake
68 screen, unknown-rate ceiling, baseline liveness), plus the control the git substrate makes possible: **re-**
69 **scoring** — an archived run's committed timeline re-measured under a changed instrument, no model
70 calls, only the instrument varying. This control has been exercised twice, on different archived runs:
71 once across an instrument change (reproducing 8 and 6 episodes exactly, localising a disagreement to
72 the runs rather than the detector), and once across the final pipeline (reproducing a bearing run's 8
73 raw / 3 declared events exactly while forty fresh runs measured zero — the evidence that the zero in
74 §5.1 belongs to the runs).

75 4 Twenty-eight ways to measure nothing

76 Every row is a real defect from this work. **S** = silent: it produced a plausible number rather than
77 an error. **G** = present while the project's test suite was green (row A4 *is* the suite, marked —). **H**
78 = findable only on a clean host, by construction. The census closes at submission; three further
79 defects caught by the §5.2 golden gate before they could produce a number are described there and
80 deliberately not counted here.

81 4.1 The catalogue, by mechanism

82 **Class A — ambient-environment dependence.** The code asks the machine a question and gets a
83 different answer on a different machine.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherit the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare python	every probe exit 127 on clean Ubuntu	✗	✓	✓
84 A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare pytest from PATH	15 tests fail on a clean host, presenting as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare python	score 0.0 manufactured by a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the pinned image	the capstone — \$5	✓	✓	✓

85 **Class B — infrastructure failure conflated with measurement.** The instrument's zero and the
86 defect's zero are the same number.

#	Defect	Consequence	S	G	H
B1	Output marker used shell : (prints nothing)	a perfect 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results fell outside the parsed slice	✓	✓	
B3	Probe diffed against a deliberately-deleted commit	every graded test a hole → "0 regressions"	✓	✓	
87 B4	Interrupted mirror clone accepted with zero commits	later instances die far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127, OCI error on stdout	✓	✓	
B6	Isolation removed loopback, not just the internet	23.5% of the oracle dead; one instance 812 tests	✓	✓	
B7	Provider cache served force-removed containers	later cells of an instance replay against a corpse and score clean	✓	✓	
B8	Per-observation tree reset reverted image build-time edits	a repo family's oracle dead, disguised as flake	✓	✓	

88 **Class C — final-state measurement of an event-shaped quantity.** The number measured is not the
89 quantity defined.

#	Defect	Consequence	S	G	H
C1	Bisection assumed monotone verdicts	recovered regressions invisible; recovering policy flattered	✓	✓	
90 C2	The declared event unit was implemented nowhere	pipeline counted per parametrised test; $\hat{\lambda}$ off by up to $2.7\times$	✓	✓	
C3	Rollbacks produced no observation	the recovering arm's recoveries invisible to the timeline scoring it	✓	✓	
C4	"Persists past the step boundary" defined nowhere	one legal reading excludes the events the design exists to count	✓	✓	

91 **Class D — output consumed without asserting its producer.**

#	Defect	Consequence	S	G	H
D1	Audit-trail field never populated	every checkpoint reported 0 tool errors, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost, scored as task failures	✗	✓	
D3	Dependency install counted as agent work	3,640 of 3,641 changed files; attribution vacuous	✓	✓	
D4	Join preferred the first observation over the last	failures pinned to the earliest tree under the fine grid	✓	✓	
D5	Executed config rebuilt from a name	manifest describes a run that never happened	✓	✓	
D6	Git handles never released	sweeps die of fd exhaustion after ~400 cells, loss concentrated on the back half	✓	✓	
D7	Console re-walked the full tree per run	answers first request, then presents as a dead server	✗	✓	
D8	Detection events never carried failing-test ids while attribution joined on them	attributed detection structurally UNKNOWN on every real run	✓	✓	
D9	Absent coverage rendered as measured silence	8 unmeasured episodes reported as attributed silence with unknown-rate 0.0	✓	✓	
D10	Turns that hit the output ceiling were executed anyway	truncated-but-parseable JSON wrote half a file; a 78-event storm read as agent incompetence, and the adapter remapped the stop reason that said why	✓	✓	

Totals, computed from the tables: **23 of 28 silent; 27 of 28 present under a green suite (the 28th being the suite itself); 6 of 28 findable only on a clean host.** D9 and D10 earn a sentence of humility: one appeared in the first valid run’s own sidecar after an audit predicted it; the other on first contact with a second model family, whose longer outputs turned a latent ceiling into a mangled tree. The class does not exhaust.

4.2 The two design rules that caught most of them

Infrastructure failure is a missing observation, never a test failure — a typed error, never fail. This converted B1, B2, B3, B5 and B6 from fabricated regressions into visible holes.

Zero events must be distinguishable from a dead instrument. The baseline-liveness gate exists because an instrument whose every probe errored once scored a perfect negative control; on first contact with a clean host it failed loudly on A2 — the defect that would otherwise have been certified as a clean result.

4.3 The working practice that found the rest

Never consume a measurement’s output without first asserting the measurement succeeded. D6 hid for weeks because a fixture consumed a sweep’s output without checking the sweep’s status. Nearly every row above is some component violating this rule; it is also how the official SWE-bench grader could be induced to mark unresolved patches resolved by forged stdout [5].

5 The capstone: every gate passed, and the number was still wrong

Three early pilots on SWE-bench Verified (80 runs, \$99.07, development slice) measured an event rate far below the pre-declared gate. The gates had been fixed in advance; re-scoring confirmed detection had not regressed; we drafted the pre-declared conclusion — switch substrates.

The conclusion was wrong. The harness materialised each task as a bare checkout on the host and ran the agent there, while every probe, verdict, and validation gate ran inside the task’s pinned image. On the host, `import matplotlib` in that checkout *succeeds* — the working directory shadows the

installed package as an uncompiled namespace package — and everything downstream fails in ways indistinguishable from agent incompetence. 26 of 28 zero-step runs traced here. We withdrew the substrate claim: the pre-declared rule behaved correctly on the data it was given; the *inference* did not survive the data’s provenance. No gate had exercised the agent’s execution path. **A validation gate certifies only the paths it exercises**, and the fix is the architecture the instrument already used for its probes: the agent must execute where it is measured.

5.1 What the corrected pipeline measures

All numbers exploratory, on a pre-declared development slice, excluded from any confirmatory frame. The declared event unit is one (test function, onset observation) pair, parametrised variants collapsed; we report **incidents** (distinct onset observations), **events** (declared unit), and **bearing runs** together, because the data are overdispersed and no single rate summarises them honestly.

Stack one — Claude planner/worker, 40 instances, rollback policy: resolve 32.5% (13/40, exact CI [18.6%, 49.1%]), and **zero events in 227 exhaustively-replayed observations** (bearing 0/40, CI [0%, 8.8%]). The same regime’s earlier two-instance canary produced one bearing run (3 declared events), so the regime’s pooled rate is near zero, not literally zero. The zero is demonstrated to belong to the runs: re-scoring that canary’s archived timeline under this identical instrument reproduces its events exactly (§3).

Stack two — GPT-5.6 planner/worker, the 10-instance prefix of the same slice, same harness, policy, and budgets: resolve 5/10 at a fifth of the cost, and **57 events across 4 incidents, concentrated in 2 of 10 runs** (bearing CI [2.5%, 55.6%]). Claude’s runs on the same 10 instances: 6/10 resolve, zero events. At $n=10$ the run-level bearing contrast is not by itself significant (Fisher exact $p \approx 0.24$, 0/10 vs 2/10); the finding is the event-level record, and the equal-N forty-instance run and the recovery-policy contrast were executing, pre-declared, at submission time. Exposure does not explain the contrast: the stacks’ observation densities differ by $1.4\times$ (8.2 vs 5.7 per run) while the event totals differ by 57-to-0 — per observation, 0.70 versus 0.00.

Two disclosures. This calibration is a re-run: the first attempt was destroyed by defect D10 (a truncated write produced a 78-event storm; the sweep’s circuit breaker stopped it at \$1.12) and is excluded as infrastructure, with the guard’s revert-verified test as the evidence the re-run is sound — no capped turns appear in its event logs. And the pre-declared gate for proceeding on this substrate still *fails* under stack two — $\hat{\lambda}$ clears its threshold but bearing $20\% < 25\%$ — so what these numbers license is not a confirmatory pass but the regime-conditional re-declaration of the gates, filed as a pre-registration amendment before the contrast arms are unblinded.

What the events were. Every one of the 57 was erased by the harness’s own rollback before the run ended; every final tree grades clean (59/59 and 145/145 on the bearing instances). So all 57 are invisible to final-state evaluation — the population prior work measures [3]. Within them, the harness’s own view splits three ways: 39 were *detected and erased* (a failing check attributable to the regression, then rollback); 16 more co-occurred with some failure without an attributable link; and — on the run that matters most — **an officially-resolved patch (all graded tests green) carried three regressions, two of which the harness never noticed at all**. One stack’s destructive edit broke 54 test functions at a single observation and left a perfect final state; a leaderboard cannot distinguish that run from one that never broke anything.

The undercount, measured across every bearing run in both sweeps:

	event-level record	visible in the final state
8 bearing runs, both sweeps	184 declared events	1

Final-state measurement — the instrument all published regression auditing uses [3] — captures 0.5% of what the timeline records here. The other 99.5% was created and repaired between the two states it examines.

5.2 The golden gate

The seam that closes A6 is validated by a *golden check*: the benchmark’s own gold patch driven through the real routed tool path at \$0 of model spend must grade resolved; a null run must not. Run

across the three hardest repo families, it passed — and caught three more defects before they could cost anything (a router aimed at a path its sandbox did not use, a stale ledger that skipped both halves while the summary read as if they ran, and a grading container inheriting the measurement’s network isolation, which would have deflated every resolve rate ~3 points forever). On the rolling benchmark we adapted next, the same gate surfaced **time-rotted oracles**: baseline tests failing in the raw image because the calendar passed dates baked into the instance — a benchmark’s instances age even as its freshness defeats contamination.

6 What this means for an agent OS

What to steal

1. **Type your silence** (§4.2, rule one) — in the schema, not in convention.
 \{ }item **Prove the instrument alive, not just non-lying** (§4.2, rule two), with positive controls that include *recovered* events.
 \{ }item **Never consume output without asserting the producer succeeded** (§4.3) — including your own ledger, fixtures, and event stream.
 \{ }item **First contact with a clean host is a validation step**. Six of our defects could not exist on the machine that wrote them.
 \{ }item **One environment**. The agent executes where it is measured, enforced by architecture. Our instrument had this seam for its probes and not for its agent; that asymmetry cost the most.

The taxonomy is the argument these rules generalise: the mechanisms are properties of subprocess-and-PATH, containers, parsers, and git — not of our code. Class A is OpenHands’ issues #4235/#7044 [6]; class D is the SWE-bench grader consuming forgeable stdout [5]; the oracle insufficiency UTBoost documents at leaderboard scale [4] is class B one level up, and the time-rotted oracles of §5.2 are its temporal form. And §5.1’s measurements give the rules a stake: the regressions that matter were *created and erased between* the states existing evaluation looks at. An OS layer that standardises recovery without standardising timeline observability will make agent runs look cleaner while hiding exactly this.

7 Related work

SWE-bench [1] and Verified [2] are the substrate. TDAD [3] measures regressions from the final patch — the design §2 argues undercounts recovered events by construction; §5.1 measures the undercount directly: 184 events, 1 final-state-visible, across every bearing run in our data. UTBoost [4] audits the oracle itself. The grader’s stdout-trust defect [5] and OpenHands’ environment issues [6] are our classes D and A in the wild. SWE-bench-Live [7] addresses contamination with rolling instances (median held-out oracle ~34× Verified’s) and exhibits the aging-oracle failure §5.2 reports. Broader evaluation-infrastructure work [8] catalogues agent-eval irreproducibility; our contribution is mechanistic: what breaks inside one instrument, which invariants stop each class, and the first event-level regression measurements on this benchmark.

8 Limitations

Everything is exploratory and dev-slice; no recovery-policy claim is made until the pre-declared contrast lands. The equal-N cross-stack comparison is a single sweep per stack — sampling variability across re-runs of the same regime is unmeasured until the seeded replications in the confirmatory plan. The regime boundary includes the provider adapter — the two stacks traverse different wire paths, which near-par resolve rates bound but cannot eliminate. The oracle observes roughly the gold test-patch’s blast radius, so "regression" here means regression in the tests adjacent to the edit; the observation hook fails open, so a missed observation thins the timeline symmetrically across stacks (its error counter is reported per cell). Binomial CIs treat the fixed dev slice as the population and ignore repository clustering (46% of the frame is one repo). The catalogue is a census of one team’s instrument; its external instances are evidence of reach, not a survey.

214 **References**

- 215 [1] Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR
216 2024. arXiv:2310.06770.
- 217 [2] OpenAI. *Introducing SWE-bench Verified*. 2024.
- 218 [3] *TDAD*. arXiv:2603.17973.
- 219 [4] Yu et al. *UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench*. ACL 2025.
220 arXiv:2506.09289.
- 221 [5] SWE-bench issue #601: *Test Result Hijacking via Stdout Forging in Evaluation Harness*.
- 222 [6] OpenHands issues #4235, #7044: environment errors in SWE-bench instance evaluation.
- 223 [7] Zhang et al. *SWE-bench Goes Live!* arXiv:2505.23419.
- 224 [8] *Holistic Agent Leaderboard*. arXiv:2510.11977.